CoRT: Code-integrated Reasoning within Thinking

Taro Yamada[1]

[1]Anonymous Institution

Abstract

Large Reasoning Models (LRMs) excel at extended chain-of-thought reasoning but remain brittle and inefficient for computationally demanding mathematical tasks. Integrating Code Interpreters (CIs) into the reasoning process can provide deterministic computation, yet models struggle with *when* and *how* to invoke tools and how to trust and incorporate execution outputs. Moreover, the field lacks high-quality training traces for code-integrated reasoning. We introduce **CoRT** (**Co**de-integrated **R**easoning within **T**hinking), a post-training framework that teaches LRMs to strategically weave code execution into their thinking process via minimal but carefully designed *hints*. CoRT employs two complementary strategies: (1) *Prompt-Hint*, a cold-start hint after `<think>` boosting code-triggering from ∼50% to ∼90%, and (2) *Hint-Engineering*, targeted cues at key junctures that trigger code during tedious computation and discourage redundant re-verification of CI outputs. From 30 curated AIME traces and 820 STILL-3 problems, CoRT synthesizes 800 SFT and 830 RFT training examples with GRPO-based RL. On five benchmarks, CoRT achieves gains of up to +4 pp over strong 32B baselines, with Prompt-Hint-SFT-32B reaching 81.8% average and Prompt-Hint-1.5B-RL surpassing DeepScaleR-1.5B-Preview at 58.3%.

1 Introduction

The emergence of Large Reasoning Models (LRMs) such as OpenAI o1 [1], DeepSeek-R1 [2], and QwQ-32B [3] has established long chain-of-thought (CoT) reasoning [4] as a powerful paradigm for complex problem solving. These models generate extended internal monologues—exploring multiple solution paths, self-correcting, and verifying intermediate steps—before arriving at a final answer. On challenging competition mathematics such as AIME, such models achieve remarkable accuracy through sheer reasoning depth.

However, LRMs remain fundamentally limited by their reliance on *text-only* computation. Precise numerical calculations, combinatorial searches, and symbolic manipulations that are trivially handled by a Python interpreter are instead performed through verbose mental arithmetic, which is both slow and error-prone. Meanwhile, it is well established that code-augmented reasoning—via Program-Aided Language Models [5], Program of Thoughts [6], or tool-integrated agents like ToRA [7]—greatly improves accuracy on mathematical tasks by offloading deterministic computation to an external executor.

The natural question is: *can we teach LRMs to seamlessly integrate code execution within their extended thinking process?* Despite recent efforts such as ReTool [8], ToRL [9], and START [10], three challenges persist. First, LRMs do not naturally call tools at the right moments—calling code too rarely or redundantly re-verifying deterministically correct CI outputs. Second, collecting high-quality interleaved training traces is expensive, requiring expert demonstration or costly RL rollouts. Third, effective RL training for tool-use requires reward design incentivizing not just correctness but efficient tool invocation.

Preprint.

We introduce **CoRT** (**Co**de-integrated **R**easoning within **T**hinking), a framework that addresses these challenges through *hint-engineered data synthesis* and a lightweight *SFT→RFT→RL* post-training pipeline. The key insight is that LRMs already possess Python coding capability—they simply need *triggering* at the right moments via minimal textual hints.

CoRT formulates reasoning as iterative text–code–execution loops within a persistent Jupyter-like environment. We propose two complementary hint strategies: (1) **Prompt-Hint**, a single general-purpose cold-start cue inserted immediately after `<think>` that globally encourages code use; and (2) **Hint-Engineering**, a collection of targeted cues that trigger code specifically when manual computation begins ("It looks tedious, and we can use python code") and suppress unnecessary re-verification of CI outputs ("We don't need to doubt the accuracy of python calculations"). These hints serve a dual purpose: they enable cold-start data collection from a capable 32B model with minimal demonstration examples, and they directly improve inference-time performance as a form of test-time scaling.

Our post-training pipeline proceeds as follows. Starting from just 30 hand-curated AIME traces, we use hint-guided rollouts over 820 STILL-3 [11] problems to produce 800 SFT traces and 830 RFT trajectories. After SFT and RFT fine-tuning of DeepSeek-R1-Distill-Qwen-32B, we distill the resulting policy into a 1.5B model and further refine it with GRPO-based RL [12], where the reward combines answer correctness with a code execution bonus ($R = R_a + \omega R_c, \omega = 0.1$) and applies output masking to stabilize training.

We evaluate CoRT on five challenging benchmarks: AIME 2024/2025, AMC23, MATH500 [13], and OlympiadBench [14]. Our main findings are:

- **32B models:** Prompt-Hint-SFT-32B achieves the best average (81.8%) among tool-augmented models; Hint-Engineering-RFT-32B attains 76.7 on AIME24 and 67.1 on AIME25 at ∼50% fewer tokens than QwQ-32B.
- **1.5B models:** Prompt-Hint-1.5B-RL reaches 58.3% average (+8 pp over SFT), surpassing DeepScaleR-1.5B-Preview.
- **Token efficiency:** CoRT reduces token usage by 30–50% across scales; the code reward adds +5% accuracy.

2 Related Work

Large Reasoning Models. Recent LRMs such as OpenAI o1 [1], o3 [15], DeepSeek-R1 [2], and QwQ-32B [3] achieve strong performance on mathematical and scientific reasoning by training models to produce long chain-of-thought reasoning before final answers. The STILL project [11] systematically studies RL-based improvements for R1-like models. Kimi k1.5 [16] and Skywork-OR1 [17] further scale RL training for reasoning. These models, however, rely exclusively on textual reasoning, limiting their precision on computation-heavy tasks.

Tool-Integrated Reasoning. Early work on tool-augmented LLMs includes Toolformer [18], which teaches models to invoke APIs via self-supervised learning, and ToRA [7], which interleaves natural language reasoning with tool calls for math problem solving. Program-Aided Language Models (PAL) [5] and Program of Thoughts [6] delegate computation entirely to code. CRITIC [19] leverages tool interaction for self-correction. More recently, integrating tools into *long-chain reasoning* has been explored: START [10] introduces hint-based inference to stimulate tool use in QwQ-32B, ReTool [8] uses RL with cold-start data to teach strategic tool invocation, ToRL [9] scales tool-integrated RL, and STILL-3-TOOL [11] demonstrates that tool manipulation dramatically boosts LRM performance. CoRT combines targeted hint engineering with a comprehensive SFT→RFT→RL pipeline.

Post-Training for Reasoning. Supervised fine-tuning with high-quality reasoning traces [20–22] and rejection sampling fine-tuning (RFT) [23, 24] improve reasoning with limited data. RAFT [25] and ReFT [26] combine reward-ranked fine-tuning with RL. GRPO [12] is a memory-efficient group-based policy optimization algorithm used in DeepSeekMath and DeepSeek-R1. DeepScaleR [27] scales RL for 1.5B models to surpass O1-Preview. CoRT builds on these advances for code-integrated reasoning.

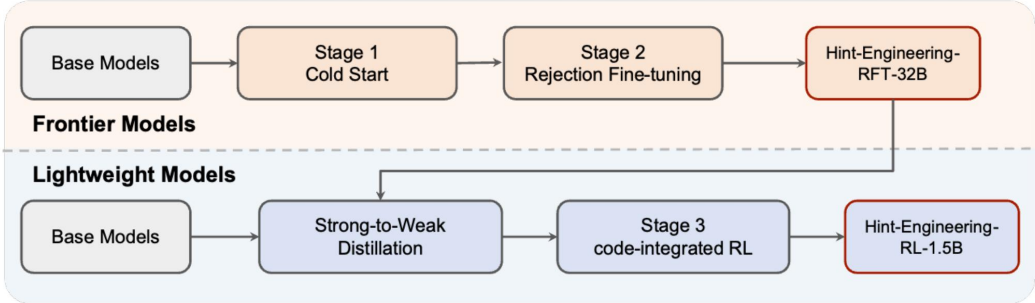


Figure 1: The CoRT training framework. Starting from a small set of cold-start exemplars, hint-guided rollouts generate SFT and RFT training data. After supervised training, GRPO-based RL with a composite reward refines the 1.5B distilled model into an efficient code-integrated reasoner.

3 Method: CoRT

3.1 Problem Formulation

We formulate code-integrated reasoning as a sequence of interleaved text and code segments within a persistent execution environment. Given a problem x, the model generates a reasoning trajectory $\tau = (s_1, c_1, o_1, s_2, c_2, o_2, \ldots, s_n, a)$, where s_i denotes a natural language reasoning step, c_i an optional Python code block, o_i the corresponding execution output returned by the CI, and a the final answer. The model operates in a Jupyter-like persistent environment: variables defined in earlier code blocks remain accessible in subsequent ones, enabling multi-step computation where each step builds on previous results.

The key challenge is that base LRMs trained without tool use rarely invoke CIs spontaneously—even when computation would clearly benefit from external execution. We address this cold-start problem through targeted hint injection.

3.2 Hint Strategies

We propose two complementary strategies for encouraging code-integrated reasoning, as illustrated in Figure 2.

Prompt-Hint. The simplest approach inserts a single, general-purpose cold-start hint immediately after the `<think>` token: "*Okay, let's try to solve this problem step by step using multiple python code calls.*" This boosts code-triggering from ∼50% to ∼90%, globally encouraging code use without specifying *when*.

Hint-Engineering. While Prompt-Hint improves code frequency, it does not ensure code is used *strategically*—at the moments where it provides the most benefit. Hint-Engineering addresses this by inserting additional targeted cues at specific reasoning junctures:

- **Computation trigger:** When tedious manual calculation begins, a hint "*It looks tedious, and we can use python code to handle this*" redirects the model to code.
- **Trust reinforcement:** After CI execution, "*We don't need to doubt the accuracy of python calculations*" discourages redundant re-verification, a common failure mode in tool-augmented LRMs.
- **General encouragement:** The Prompt-Hint cue is also inserted after `<think>` to establish a code-friendly disposition.

These hints are inserted during *inference-time rollouts* to collect training data and can also be used directly as test-time hints for zero-shot improvement.

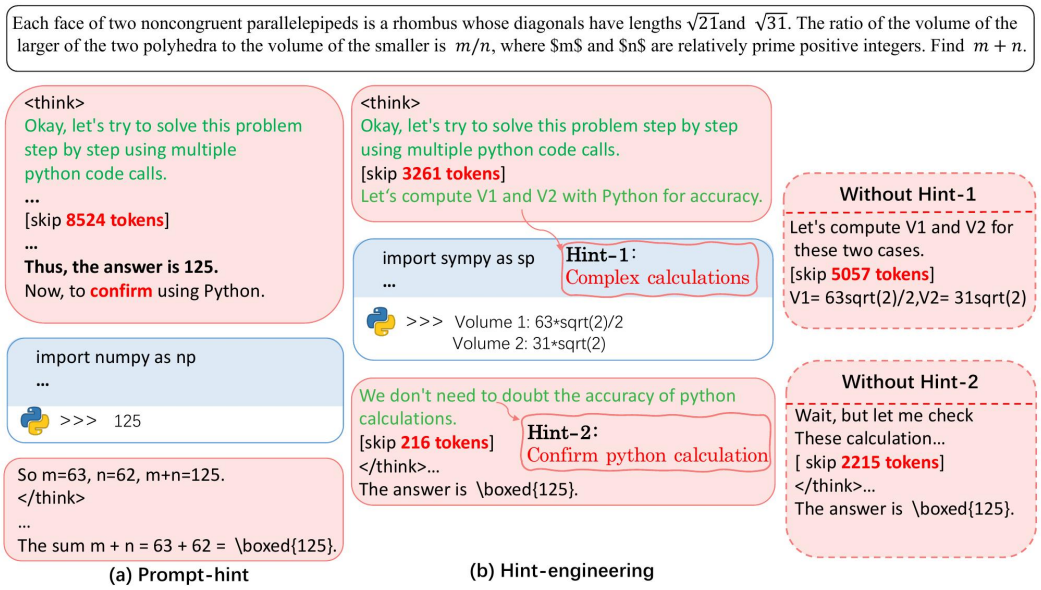


Figure 2: Comparison between Prompt-Hint and Hint-Engineering approaches on AIME 2023 I Problem 13 (prompt prefix omitted for brevity). Both begin with a general hint (green) after `<think>`, but Hint-Engineering additionally inserts targeted cues at key reasoning junctures to trigger code at computation-heavy steps and discourage redundant re-verification of CI outputs.

3.3 Data Synthesis Pipeline

Cold-start exemplars. We manually curate 30 high-quality AIME problem solutions demonstrating fluent code-integrated reasoning—where code is invoked at precisely the right moments, execution outputs are trusted and built upon, and the reasoning is concise and coherent. These 30 exemplars serve as demonstrations for few-shot prompting to initialize the rollout policy.

SFT data collection. We apply hint-guided inference to the DeepSeek-R1-Distill-Qwen-32B model on 820 problems from the STILL-3 dataset [11], using the 30 AIME exemplars as few-shot demonstrations. We collect trajectories where the model correctly solves the problem, filtering for quality and diversity of code usage patterns. This yields approximately 800 SFT training examples with interleaved reasoning and code.

RFT data augmentation. We further apply rejection sampling fine-tuning (RFT) [23]: using the hint-steered model, we generate multiple trajectories per problem and retain correct ones, yielding approximately 830 additional RFT training examples with diverse reasoning paths.

3.4 Post-Training Pipeline

The CoRT training pipeline (Figure 1) proceeds in three stages: SFT, RFT, and RL.

Stage 1: Supervised Fine-Tuning. We fine-tune DeepSeek-R1-Distill-Qwen-32B on the 800 SFT examples using standard cross-entropy loss over the full trajectory, including both reasoning tokens and code blocks. The persistent execution environment is simulated by concatenating code blocks and their outputs in order within the context.

Stage 2: Rejection Sampling Fine-Tuning. The SFT model generates additional trajectories on the training problems; correct ones are added to the training set and the model is further fine-tuned [23, 24], particularly benefiting harder problems where the SFT model succeeds only occasionally.

Stage 3: Reinforcement Learning with GRPO. For the 1.5B model, we apply Group Relative Policy Optimization (GRPO) [12] after distilling the 32B policy into DeepSeek-R1-Distill-Qwen-

1.5B. GRPO samples groups of completions for each problem and computes advantages relative to the group mean reward, avoiding the need for a separate value function.

Reward design. We use a composite reward:

$$R = R_a + \omega R_c, \quad \omega = 0.1 \tag{1}$$

where $R_a \in \{0, 1\}$ is the answer correctness reward (verified by rule-based matching) and $R_c \in [0, 1]$ is a code execution reward that measures whether code blocks are syntactically valid, execute without errors, and produce results that are used in the reasoning. The code reward encourages the model to write functional, integrated code rather than inert or isolated snippets.

Output masking. To stabilize RL training and prevent the model from gaming the reward by producing verbose or repetitive text, we apply output masking: the policy gradient is computed only over model-generated tokens, excluding CI execution outputs (which are deterministic given the code) and the injected hint tokens themselves.

Maximum sequence and tool use limits. All experiments use a maximum sequence length of 32,768 tokens and limit tool usage to a maximum of 15 code calls per problem, following standard practice [8, 11].

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate on five challenging mathematical reasoning benchmarks:

- **AIME 2024 & 2025**: American Invitational Mathematics Examination, competition-level problems requiring deep mathematical insight.
- **AMC 2023**: American Mathematics Competition, covering a broad range of competition math topics.
- **MATH500** [13, 28]: A 500-problem subset of the MATH benchmark at the highest difficulty levels.
- **OlympiadBench** [14]: Olympiad-level bilingual mathematical problems.

Inference setup. We use temperature 0.6 and top-p 0.95. Results for AIME24, AIME25, and AMC23 are averaged over 16 samples; MATH500 and OlympiadBench over 4 samples. Answer verification uses Math-Verify [29].

Baselines. We compare against: (i) SOTA models without tool use (o1 [1], DeepSeek-R1 [2], QwQ-32B [3]); (ii) frontier 32B models without tool use (DeepSeek-R1-32B); and (iii) tool-augmented models (START [10], STILL-3-TOOL [11], ReTool [8], ToRL [9], DeepScaleR [27]).

4.2 Main Results

Table 1 presents the main comparison across all benchmarks. We make the following observations:

32B models. Prompt-Hint-SFT-32B achieves the best average accuracy (81.8%) among all tool-augmented models, outperforming STILL-3-TOOL-32B (81.0%), ReTool-R1-32B (76.6%), and the non-tool DeepSeek-R1-32B baseline (77.5%). Notably, it attains 77.3 on AIME24, 65.0 on AIME25, 95.0 on AMC23, 96.6 on MATH500, and 75.1 on OlympiadBench. Hint-Engineering-RFT-32B achieves competitive results (81.3% average), with 76.7 on AIME24 and a particularly strong 67.1 on AIME25—the highest among all listed models on this benchmark. These results demonstrate consistent, broad gains over the base model (+4 pp average).

1.5B models. At the smaller scale, Prompt-Hint-1.5B-RL achieves 58.3% average accuracy, surpassing both DeepScaleR-1.5B-Preview (56.3%) and ToRL-1.5B (48.5%). Critically, this represents a +8 pp improvement over the Prompt-Hint-1.5B-SFT baseline (50.5%), demonstrating the effectiveness of RL fine-tuning. Hint-Engineering-1.5B-RL achieves 56.4%, also competitive with DeepScaleR.

Table 1: Performance comparison of different math reasoning models across benchmarks. For each section, best results are shown in **bold** and second-best results are underlined. During inference, we set temperature 0.6 and top_p 0.95. Results for AIME24, AIME25, and AMC23 are averaged over 16 samples, while MATH500 and Olympiad results are averaged over 4 samples. All experiments use a maximum sequence length of 32,768 tokens and limit tool usage to 15 calls.

Model	Tool-Use	AIME24	AIME25	AMC23	MATH500	Olympiad	Avg
SOTA Models							
o1	✗	74.3	**79.2**	-	96.4	-	-
DeepSeek-R1	✗	**79.8**	70.0	-	**97.3**	-	-
QwQ-32B	✗	79.5	65.3	**94.3**	92.3	**79.7**	**82.2**
Frontier Models (32B)							
DeepSeek-R1-32B	✗	72.9	59.0	88.8	94.3	72.5	77.5
START-32B	✓	66.7	47.1	**95.0**	94.4	-	-
STILL-3-TOOL-32B	✓	76.7	64.4	91.3	**96.6**	**75.9**	81.0
ReTool-R1-32B	✓	72.5	54.3	92.9	94.3	69.2	76.6
Prompt-Hint-SFT-32B	✓	**77.3**	65.0	**95.0**	**96.6**	75.1	**81.8**
Hint-Engineering-SFT-32B	✓	72.1	60.2	91.3	94.4	71.2	77.8
Hint-Engineering-RFT-32B	✓	76.7	**67.1**	94.4	95.1	73.4	81.3
Lightweight Models (1.5B)							
DeepSeek-R1-1.5B	✗	28.8	21.8	62.9	83.9	43.3	48.1
DeepScaleR-1.5B-Preview	✗	40.0	30.0	73.6	**87.8**	50.0	56.3
ToRL-1.5B	✓	26.7	26.7	67.5	77.8	44.0	48.5
Prompt-Hint-1.5B-SFT	✓	30.6	25.0	63.1	83.3	50.4	50.5
Prompt-Hint-1.5B-RL	✓	**43.1**	**30.2**	**73.8**	87.3	**57.1**	**58.3**
Hint-Engineering-1.5B-SFT	✓	34.0	23.5	64.6	84.2	49.8	51.2
Hint-Engineering-1.5B-RL	✓	41.0	29.4	70.0	85.8	55.6	56.4

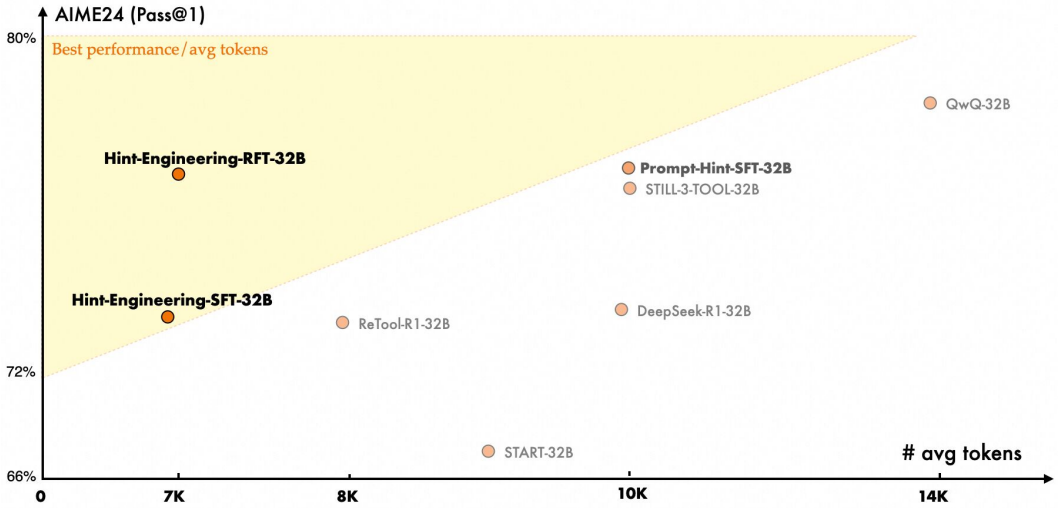
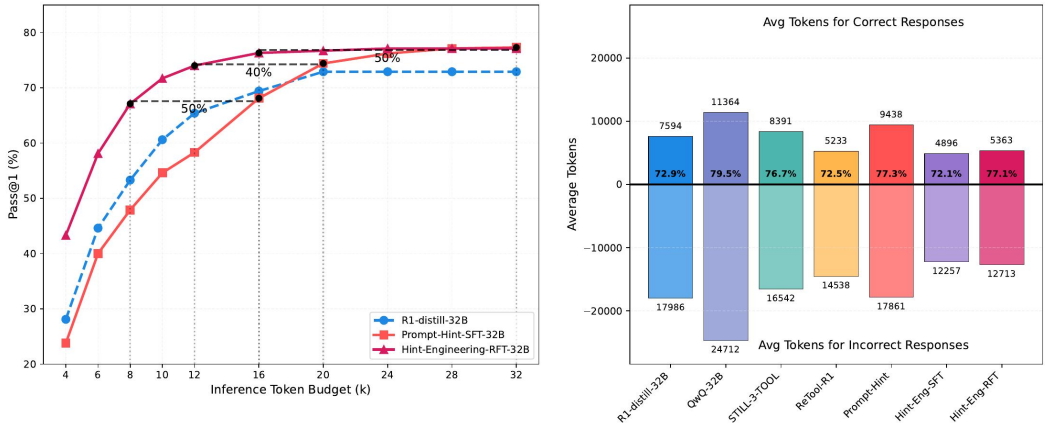


Figure 3: Performance vs. token usage comparison on AIME24. CoRT models (particularly Hint-Engineering-RFT-32B) achieve performance comparable to QwQ-32B while using approximately 50% fewer tokens, demonstrating substantially improved token efficiency.

These results validate that CoRT's pipeline successfully distills code-integrated reasoning capability into small models.

4.3 Token Efficiency Analysis

A key advantage of code-integrated reasoning is that it replaces lengthy mental arithmetic with concise, precise code execution. Figures 3 and 4 illustrate this efficiency gain. Hint-Engineering-RFT-32B matches QwQ-32B performance on AIME24 (76.7 vs. 79.5) while consuming approximately



(a) Performance on AIME24 with different token budget (b) Average token usage on AIME24

Figure 4: Detailed token efficiency analysis on AIME24. CoRT models achieve comparable or higher accuracy with significantly fewer tokens compared to text-only baselines, demonstrating that code execution replaces verbose mental arithmetic.

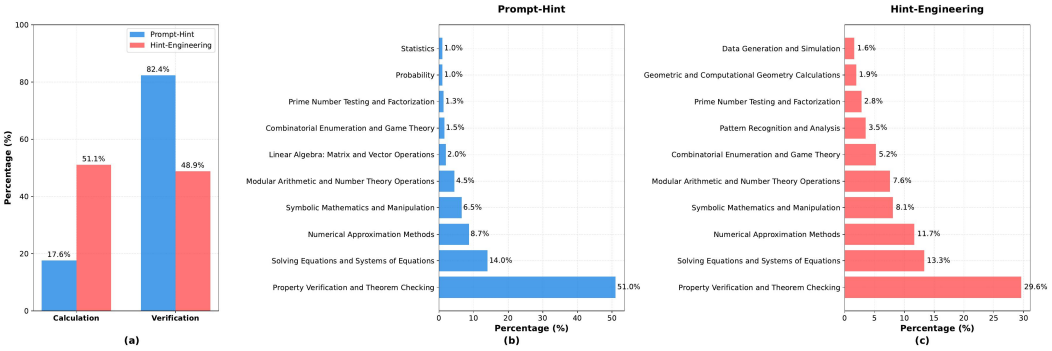


Figure 5: Analysis of Python code usage patterns across models and training stages. CoRT models show a behavioral shift from verification-heavy code use (common in base models that distrust CI outputs) to balanced calculation and verification, with code invoked primarily for computation rather than redundant checking.

50% fewer tokens (7K vs. 14K tokens per problem). At the 1.5B scale, CoRT models achieve roughly 50% token reduction compared to non-tool baselines. This efficiency stems from the model learning to delegate computation to the CI rather than performing it step-by-step in text, resulting in more concise and structured reasoning chains.

4.4 Analysis of Code Usage Patterns

Figure 5 analyzes how CoRT models use Python code compared to baselines. Several behavioral shifts emerge:

1. **Higher code frequency:** Hint-steered models invoke code on a much higher fraction of problems ($\sim$90% with Prompt-Hint vs. $\sim$50% for the base model).
2. **Computation-first usage:** Hint-Engineering models primarily use code for computation (setting up equations, performing arithmetic, searching over cases), rather than re-verifying already-known results.
3. **Reduced verification overhead:** The "We don't need to doubt the accuracy of python calculations" hint successfully conditions models to trust CI outputs, eliminating wasted tokens on re-verification.

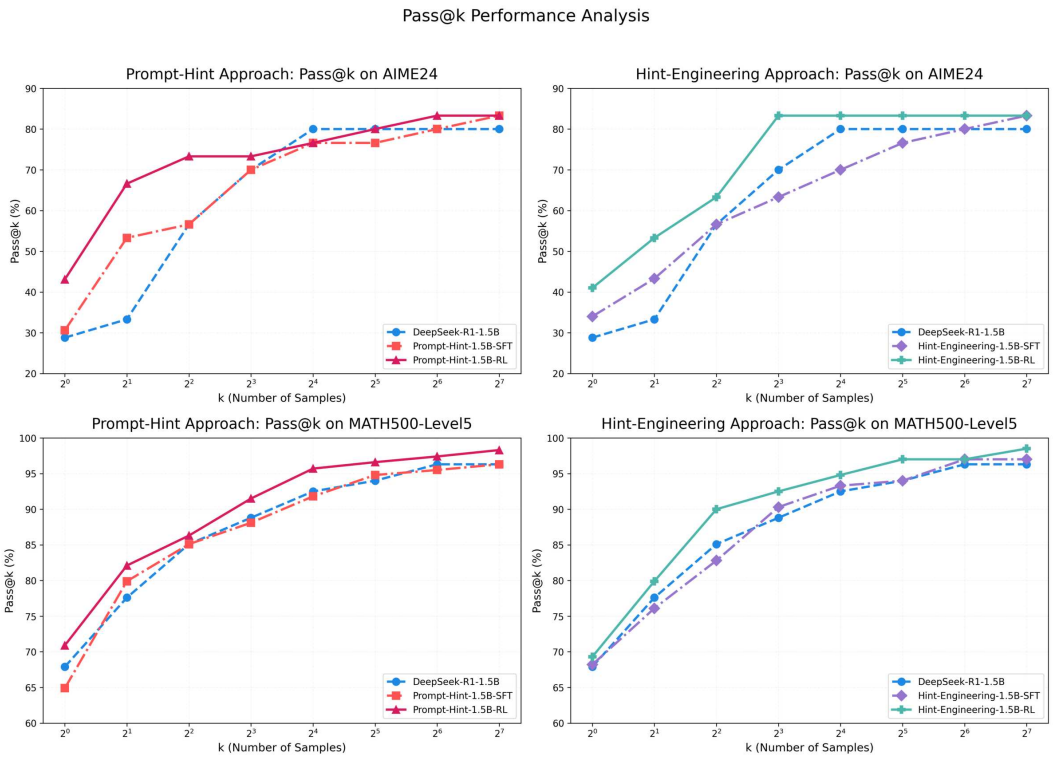


Figure 6: Pass@k performance on AIME24 (top) and MATH500-Level5 (bottom) for Prompt-Hint (left) and Hint-Engineering (right) approaches. CoRT models show strong diversity in reasoning strategies across samples, with both approaches achieving high pass@k, indicating broad coverage of solution methods.

4. **Multi-step integration:** Models trained with CoRT naturally build multi-step computations across code blocks, exploiting the persistent execution environment to incrementally refine solutions.

This behavioral shift from verification-heavy to calculation-oriented code use is a key contributor to CoRT's token efficiency and accuracy gains.

4.5 Pass@k Analysis

Figure 6 shows CoRT models exhibit high diversity across samples, yielding strong pass@k and broad solution-space coverage.

4.6 Ablation Studies

Code execution reward. Figure 7 ablates the effect of the code execution reward component R_c during RL training. Training with the composite reward ($R = R_a + 0.1 \cdot R_c$) consistently outperforms training with only the answer reward ($R = R_a$) across AIME24 performance levels. The code reward provides two benefits: it directly incentivizes syntactically valid and executable code, and it stabilizes training by providing denser learning signal for problems where the final answer alone is uninformative (e.g., when the model writes code that runs correctly but arrives at a wrong final answer). Across experiments, $\omega = 0.1$ provides up to +5% accuracy improvement over $\omega = 0$.

SFT vs. RFT vs. RL. Comparing the three training stages at 32B scale: Hint-Engineering-SFT-32B (77.8% avg) $<$ Hint-Engineering-RFT-32B (81.3% avg), demonstrating that rejection sampling augmentation substantially improves performance. At 1.5B scale: Prompt-Hint-1.5B-SFT (50.5%) $<$ Prompt-Hint-1.5B-RL (58.3%), confirming that RL training is essential for maximizing performance of smaller models.

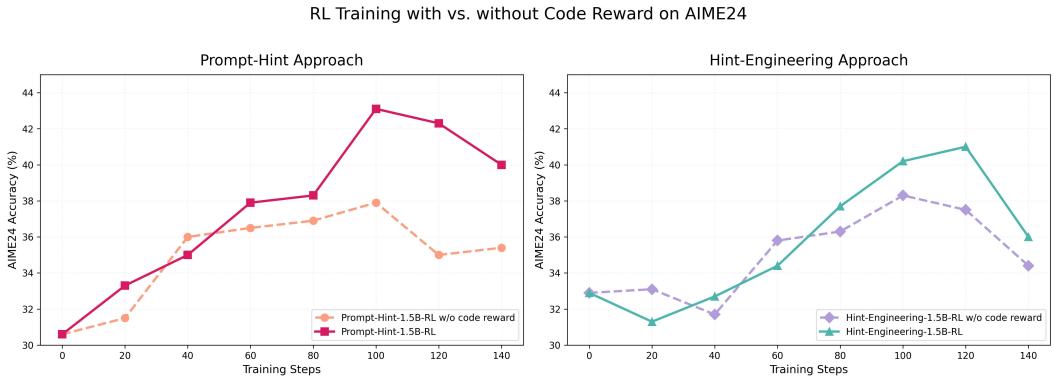


Figure 7: Ablation study on the impact of the code execution reward R_c during RL training on AIME24. Including the code reward ($\omega = 0.1$) consistently improves both accuracy and code usage frequency compared to training with answer reward alone.

Prompt-Hint vs. Hint-Engineering. At 32B scale, Prompt-Hint-SFT-32B (81.8%) marginally outperforms Hint-Engineering-RFT-32B (81.3%) in average accuracy, with Prompt-Hint showing stronger absolute performance on most benchmarks. However, Hint-Engineering achieves notably better performance on AIME25 (67.1 vs. 65.0), suggesting the targeted cues help on particularly challenging problems. At 1.5B scale, Prompt-Hint-1.5B-RL (58.3%) outperforms Hint-Engineering-1.5B-RL (56.4%), suggesting Prompt-Hint's simpler inductive bias is easier to transfer to smaller models.

5 Discussion

Why hints work. Pre-trained LRMs already understand Python syntax—they simply lack the in-context signal to invoke code. Hints resolve this, requiring just 30 manually curated AIME traces to bootstrap the pipeline.

Limitations. CoRT's current evaluation focuses on mathematical problem solving with Python code interpreters. Extending to other domains (e.g., scientific reasoning, coding tasks) and other tool types (web search [30, 31], databases) is an important direction for future work. Additionally, the hint design process currently involves manual engineering; automating hint discovery (e.g., via meta-learning or RL over hint strategies) could further reduce human effort.

6 Conclusion

We presented CoRT, a framework for teaching Large Reasoning Models to strategically integrate code execution within their extended thinking. CoRT's core contribution is hint-engineered data synthesis leveraging just 30 manually curated traces to bootstrap high-quality SFT and RFT training data, followed by GRPO-based RL with a composite code-execution reward. Through two complementary hint strategies—Prompt-Hint for global code encouragement and Hint-Engineering for targeted computation triggers—CoRT produces models that invoke code when most beneficial and trust CI outputs without redundant re-verification.

Across five benchmarks, CoRT achieves consistent gains at both scales while reducing token consumption by 30–50%, with behavioral analysis confirming a shift from verification-heavy to computation-oriented code use.

References

[1] OpenAI. Learning to reason with llms. *https://openai.com/index/learnin g-to-reason-with-llms/*, 2024.

[2] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*, abs/2501.12948, 2025.

[3] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025. URL https://qwenlm.github.io/blog/qwq-32b/.

[4] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[5] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pages 10764–10799. PMLR, 2023.

[6] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.

[7] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. Tora: A tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv:2309.17452*, 2023.

[8] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. Retool: Reinforcement learning for strategic tool use in llms, 2025. URL https://arxiv.org/abs/2504.11536.

[9] Kezhou Wang, Ruijie Wu, Qinlin Zeng, Huao Lu, Hanye Wu, Qingfeng Cui, Haichao Lin, Yujia Liu, Xiaoyan Huang, Qingpeng Guo, Songtao Jian, Kaiyuan Lu, Shiyu Li, Hao Tian, Yongqin Sun, Xue Yang, Libin Song, Zejun Ou, and Guoqing Wang. ToRL: Scaling tool-integrated RL for LLMs. *arXiv preprint arXiv:2312.10372*, 2023.

[10] Chengpeng Li, Mingfeng Xue, Zhenru Zhang, Jiaxi Yang, Beichen Zhang, Xiang Wang, Bowen Yu, Binyuan Hui, Junyang Lin, and Dayiheng Liu. Start: Self-taught reasoner with tools. *arXiv preprint arXiv:2503.04625*, 2025.

[11] Zhipeng Chen, Yingqian Min, Beichen Zhang, Jie Chen, Jinhao Jiang, Daixuan Cheng, Wayne Xin Zhao, Zheng Liu, Xu Miao, Yang Lu, et al. An empirical study on eliciting and improving r1-like reasoning models. *arXiv preprint arXiv:2503.04548*, 2025.

[12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[13] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks*, 2021.

[14] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems, 2024.

[15] OpenAI. Introducing openai o3 and o4-mini, 2025. URL https://openai.com/index/introducing-o3-and-o4-mini/.

[16] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.

[17] Jujie He, Jiacai Liu, Chris Yuhao Liu, Rui Yan, Chaojie Wang, Peng Cheng, Xiaoyu Zhang, Fuxiang Zhang, Jiacheng Xu, Wei Shen, Siyuan Li, Liang Zeng, Tianwen Wei, Cheng Cheng, Bo An, Yang Liu, and Yahui Zhou. Skywork open reasoner series. https://capricious-hydrogen-41c.notion.site/Skywork-Open-Reaonser-Series-1d0bc9ae823a80459b46c149e4f51680, 2025. Notion Blog.

[18] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

[19] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. Critic: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738*, 2023.

[20] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning, 2025. URL https://arxiv.org/abs/2502.03387.

[21] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*, 2025.

[22] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

[23] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models, 2023. URL https://arxiv.org/abs/2308.01825.

[24] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

[25] Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: reward ranked finetuning for generative foundation model alignment. *Trans. Mach. Learn. Res.*, 2023.

[26] Trung Quoc Luong, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. Reft: Reasoning with reinforced fine-tuning. *arXiv preprint arXiv:2401.08967*, 3, 2024.

[27] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2, 2025. Notion Blog.

[28] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[29] Hynek Kydlíček. Math-verify: Math verification library, 2024. URL https://github.com/huggingface/math-verify.

[30] Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. Search-o1: Agentic search-enhanced large reasoning models, 2025. URL https://arxiv.org/abs/2501.05366.

[31] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2503.05592.